

AgenteRevisa

SISTEMA MULTIAGENTE DE REVISÃO DE CÓDIGO JAVA

Nathalia Barbosa
NFAB

Thales Bezerra
TOB2

Vinicius Barbosa
VRB

— Background

- Pipelines de CI/CD frequentemente rodam ferramentas de análise estática e linting quando PRs são abertos, como o Spotbugs, PMD e Checkstyle;
- Quando essas ferramentas indicam erros, os desenvolvedores precisam corrigir parte deles manualmente;
- Nossa ferramenta busca automatizar esse processo de refatoração, executando as ferramentas e fazendo refatorações que buscam preservar a lógica inicial do código.

O que o *AgenteRevisa* faz?

- Executa as ferramentas de análise estática Spotbugs, PMD e Checkstyle;
- Resolve os problemas identificados no código;
- Reexecuta iterativamente as ferramentas, para garantir que as alterações não introduziram outros problemas;
- Gera testes que são executados antes e depois das alterações, e servem para garantir que as regras de negócio não serão quebradas pelas mudanças feitas.

Ferramentas Suportadas

JUnit 

Geração de
Testes

 SpotBugs

SpotBugs



PMD

 checkstyle

CheckStyle

Ferramentas Suportadas



Geração de
Testes

- O Orquestrador chama o agente gerador de testes primeiro, antes das ferramentas;
- O Agente pode gerar testes e executá-los. Em caso de erros (compilação ou execução), pode consertá-los;
- Os testes auxiliam na preservação da lógica de negócio ao longo da execução dos outros agentes.

Ferramentas Suportadas



SpotBugs

- O SpotBugs verifica mais de 400 padrões de bugs em códigos compilados. É usado para detectar problemas sem precisar executar o código.
- O Agente recebe o código, os testes e os erros apontados pelo SpotBugs. É esperado o código alterado como saída.

Ferramentas Suportadas



PMD

- A ferramenta encontra falhas comuns, como variáveis não utilizadas, blocos vazios e criação desnecessária de objetos
 - Os erros detectados dependem dos *rulesets* usados: nós usamos o “quickstart”, que contém as regras mais gerais.
- O Agente recebe o código, os testes e os erros apontados pelo PMD. É esperado o código alterado como saída.

Ferramentas Suportadas

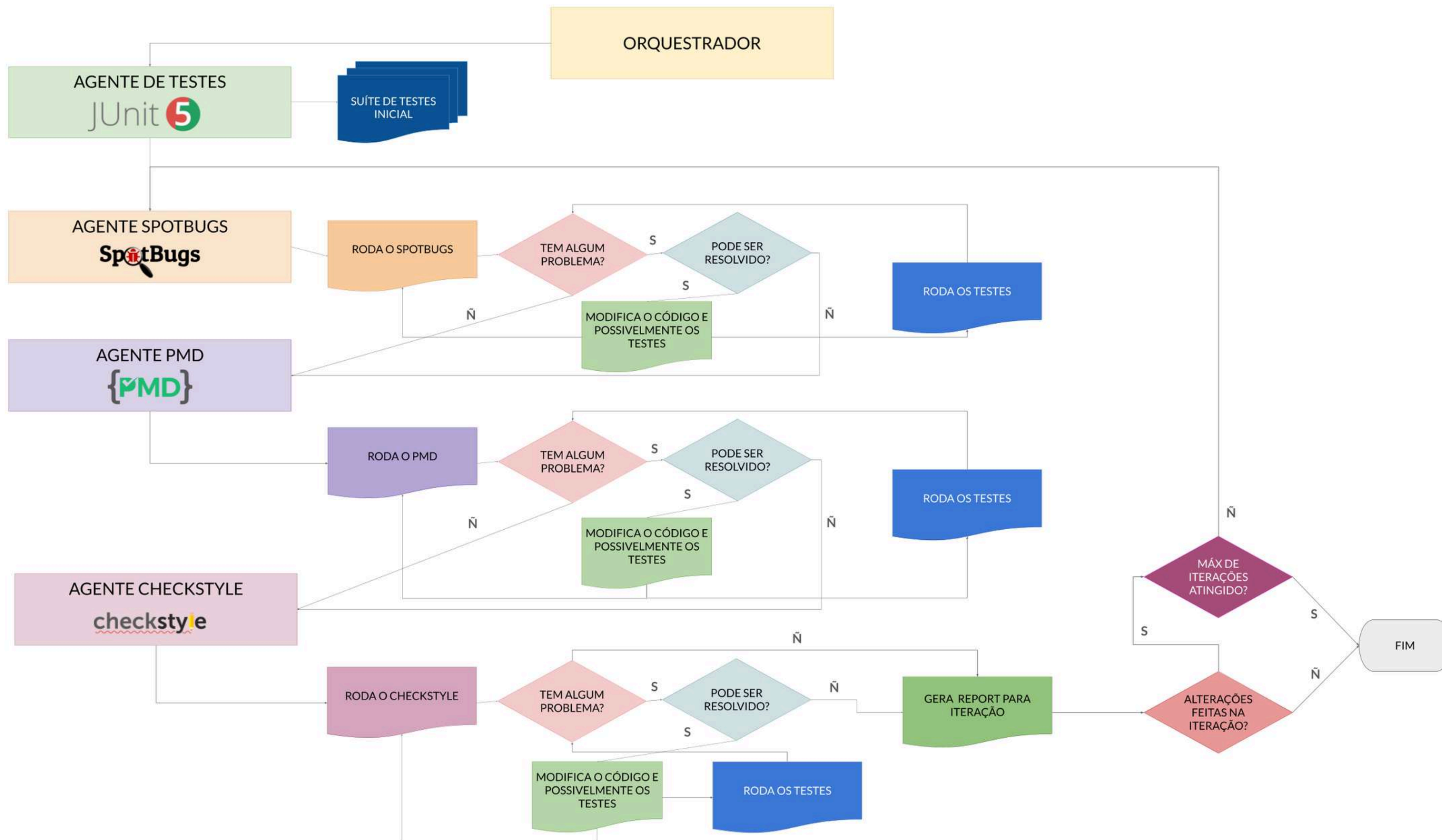
The logo for Checkstyle, featuring the word "checkstyle" in a bold, sans-serif font. The "e" is stylized with a yellow pencil tip and a red squiggly line underneath the word.

CheckStyle

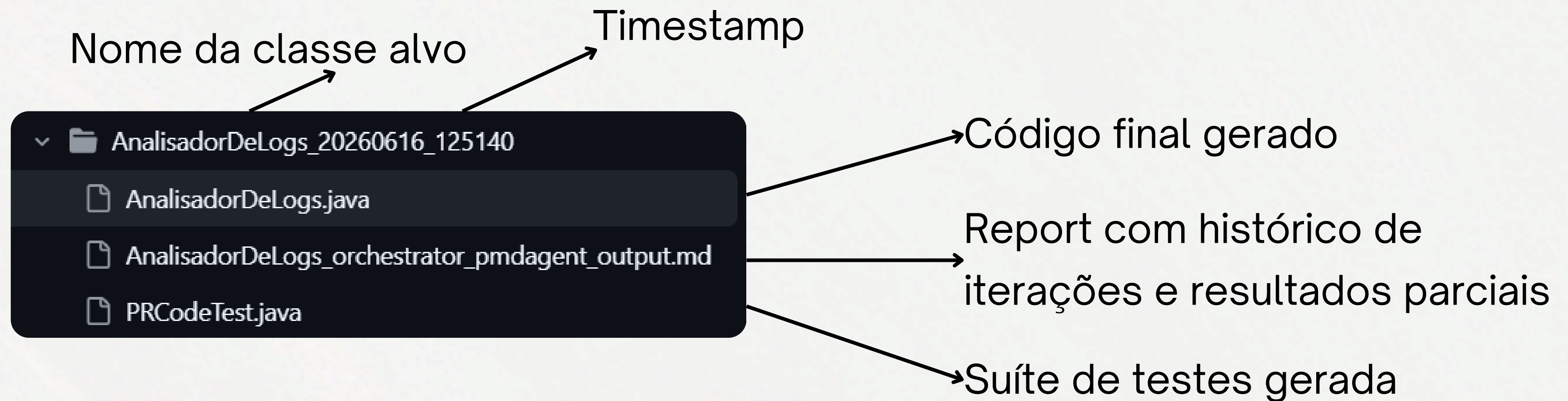
- O Checkstyle é uma ferramenta para seguir um padrão de estilo, padronizando formatação e indentação, por exemplo. A convenção usada (Sun Code Style, Google Java Style) é escolhida pelo Agente.
- O Agente recebe o código, os testes e os erros apontados pelo CheckStyle. É esperado o código alterado como saída.

— Implementação

- Ferramenta chamada por CLI, implementada com LangGraph e com suporte a API do Gemini.
- **Nos Agentes:**
 - Prompt Chaining,
 - Multi-Agent Collaboration,
 - Routing,
 - Goal Setting and Monitoring
- **Nos *prompts*:** uso de *roles* e *system prompting*, definir formato de saída, delimitadores, não uso de *constraints*.



Formato de Saída



Avaliação

- 1 Criação de um *dataset* com classes que contêm problemas detectados por cada ferramenta (10 problemas cada → 30 no total)
- 2 Execução dos agentes nas classes do *dataset* usando o modelo Gemini 3.1 Flash-Lite e 3 iterações por ferramenta
- 3 Análise manual das saídas

Análise do Spotbugs

- Conseguiu corrigir os principais problemas;
- Adicionou novos trechos de código em alguns exemplos que quebram retrocompatibilidade.

Spotbugs: Exemplo de falha

- Adicionou do nada uma trava com `if (valorMaximo <= 0) throw new IllegalArgumentException(...)`
- Antes, se alguém mandasse 0 ele só devolveria 0. Agora, lança exceção em *runtime* e derruba a aplicação.

Antes

```
public int gerarIdLoteAleatorio(int valorMaximo) {  
    return (int) (random.nextDouble() * valorMaximo);  
}
```

Depois

```
public int gerarIdLoteAleatorio(int valorMaximo) {  
    if (valorMaximo <= 0) {  
        throw new IllegalArgumentException("0 valor maximo deve ser positivo.");  
    }  
    return random.nextInt(valorMaximo);  
}
```

Análise do PMD

- Conseguiu corrigir os principais problemas em poucas iterações
- Causou alguns problemas ocasionais, como geração de código desnecessário e refatoração de partes que não precisavam ser mudadas (coisas não alertadas pelo PMD).
- Em um dos casos, o LLM deixou comentários em vez de implementar o código em alguns trechos. (“// Lógica aqui...”)

PMD: Exemplo de falha

- Transformou a interface numa classe e criou um construtor privado.

Antes

```
package com.ficticio.pmd;

public interface StatusDoPedidoConstantes {

    String STATUS_PENDENTE = "PENDENTE";

    String STATUS_PAGO = "PAGO";

    String STATUS_CANCELADO = "CANCELADO";

    String STATUS_ENVIADO = "ENVIADO";

}
```

Depois

```
package com.ficticio.pmd;

public final class StatusDoPedidoConstantes {

    public static final String STATUS_PENDENTE = "PENDENTE";
    public static final String STATUS_PAGO = "PAGO";
    public static final String STATUS_CANCELADO = "CANCELADO";
    public static final String STATUS_ENVIADO = "ENVIADO";

    private StatusDoPedidoConstantes() {
        // Construtor privado para evitar instanciação
    }

}
```

Análise do CheckStyle

- Conseguiu corrigir os principais problemas;
- Fez muitas mudanças com base apenas no contexto, sem apoio do report da ferramenta (*não é necessariamente ruim*).

CheckStyle: Exemplo de falha

- Renomeou vários parâmetros sem ser solicitado (inventando nomes).
- A documentação gerada se baseou nesses nomes inventados.

Antes

```
public void avaliar(int parametro1, String parametro2, double parametro3, boolean parametro4, String parametro5) {  
    if (parametro1 >= 18) {
```

Depois

```
/**  
 * Avalia se o usuário atende aos requisitos necessários.  
 *  
 * @param idade idade do usuário  
 * @param status status do usuário  
 * @param saldo saldo do usuário  
 * @param ativo indicador de atividade  
 * @param categoria categoria do usuário  
 */  
public void avaliar(int idade, String status, double saldo, boolean ativo, String categoria) {  
    if (idade < 18) {
```

Trabalhos Relacionados

- **RepairAgent: An Autonomous, LLM-Based Agent for Program Repair (*ICSE 2025*)** - usam o GPT-3.5 na tarefa de consertar bugs do Defects4J com agentes
- **Agentic Program Repair from Test Failures at Scale: A Neuro-symbolic approach with static analysis and test execution feedback (*TSE 2026*)** - usam modelos Llama com uma abordagem ReAct para corrigir código baseado em falhas de testes em monorepos da Meta, refinando as soluções iterativamente com feedback de análise estática e execução de testes
- **GitHub Copilot Code Review**

Obrigado!

[HTTPS://GITHUB.COM/NATHALIAFAB/AGENTEREVISA](https://github.com/nathaliafab/AGENTEREVISA)